Experiment No-30 : Backward Chaining

Aim:

To write a Prolog program to implement Backward Chaining and incorporate the required queries.

Programming Code:

```
% Backward chaining: start from the goal and work backwards, trying to prove
% every condition of a rule that can conclude the goal.

% Known facts
fact(has_wheels).
fact(has_engine).
fact(has_two_wheels).
fact(has_handlebar).

% rule(Conclusion, ListOfConditions)
rule(motor_vehicle, [has_wheels, has_engine]).
rule(two_wheeler, [motor_vehicle, has_two_wheels]).
rule(motorbike, [two_wheeler, has_handlebar]).
rule(car, [motor_vehicle, has_four_wheels, has_steering]).

% prove(Goal): a goal is true if it is a fact or all conditions of a rule are true
prove(Goal) :- fact(Goal).
prove(Goal) :- rule(Goal, Conditions), prove_all(Conditions).

prove_all([]).
prove_all([H|T]) :- prove(H), prove_all(T).

% backward(Goal): same as prove/1 but prints the reasoning steps
backward(Goal) :- bc(Goal, 0).

bc(Goal, D) :-
    fact(Goal),
    tab(D), format("~w is a known fact~n", [Goal]).
bc(Goal, D) :-
    rule(Goal, Conditions),
    tab(D), format("To prove ~w, prove ~w~n", [Goal, Conditions]),
    D1 is D + 2,
    bc_all(Conditions, D1),
    tab(D), format("~w is proved~n", [Goal]).

bc_all([], _).
bc_all([H|T], D) :- bc(H, D), bc_all(T, D).
```

OUTPUT:

```
$ swipl -q exp30.pl

?- prove(motorbike).
true.

?- backward(motorbike).
To prove motorbike, prove [two_wheeler,has_handlebar]
  To prove two_wheeler, prove [motor_vehicle,has_two_wheels]
    To prove motor_vehicle, prove [has_wheels,has_engine]
      has_wheels is a known fact
      has_engine is a known fact
    motor_vehicle is proved
    has_two_wheels is a known fact
  two_wheeler is proved
  has_handlebar is a known fact
motorbike is proved
true.

?- backward(car).
To prove car, prove [motor_vehicle,has_four_wheels,has_steering]
  To prove motor_vehicle, prove [has_wheels,has_engine]
    has_wheels is a known fact
    has_engine is a known fact
  motor_vehicle is proved
false.

?- prove(car).
false.

?- prove(G).
G = has_wheels ;
G = has_engine ;
G = has_two_wheels ;
G = has_handlebar ;
G = motor_vehicle ;
G = two_wheeler ;
G = motorbike.

$
```

Result

Backward Chaining was implemented successfully using Prolog. Starting from the goal, the program works backwards through the rules and facts. The goal motorbike is proved, while the goal car fails because the facts has_four_wheels and has_steering are not known.